UC-001 Ask a question and get a grounded answer

Ask a question and get a grounded answer

Use case, reversed from the implementation at 0.3.0. Draft.

The solution in one sentence

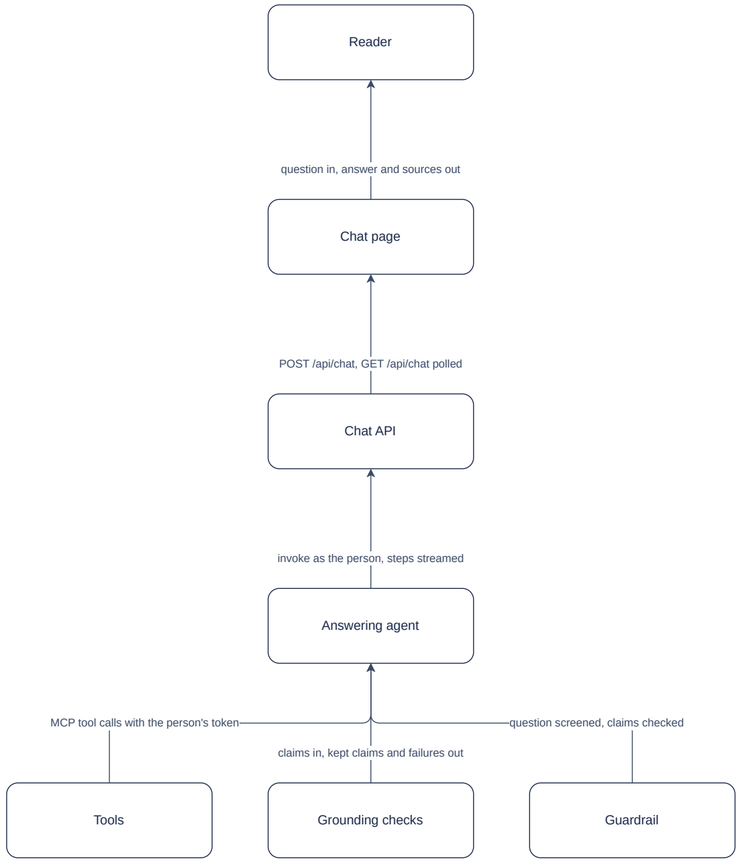
A reader asks a question of a collection in plain language and gets an answer made only of statements that link to the passage or the table cell they came from, so that every sentence can be checked by opening its source.

Documents answer questions slowly and a model answers them fluently but unverifiably. The reader needs the speed of the second with the authority of the first.

What is different afterwards

The reader clicks a numbered citation and sees the quoted words highlighted in the passage, or the row of the table with the cell highlighted. When the sources cannot answer, the reader is told so and what was missing, and can ask what it would take.

The solution on one page



Generated from the Participants and Interactions tables with the model skill; `components.drawio` is the source.

Participants

Participant	Role in this use case	Kind
01 Reader	Asks, reads, checks	actor
02 Chat page	Takes the question, shows claims, sources, steps	solution
03 Chat API	Checks quota, dispatches, keeps the answer	solution
04 Answering agent	Plans in the ontology's terms, calls tools, proposes claims	solution
05 Tools	Fixed queries over the graph, tables and passages, as the caller	solution
06 Grounding checks	Re-reads every citation and keeps what passes	solution
07 Guardrail	Screens the question; checks each claim's grounding	external

Interactions

Interface	Provider	Consumer	Purpose
IF-01	02 Chat page	01 Reader	question in, answer and sources out
IF-02	03 Chat API	02 Chat page	POST /api/chat, GET /api/chat polled
IF-03	04 Answering agent	03 Chat API	invoke as the person, steps streamed
IF-04	05 Tools	04 Answering agent	MCP tool calls with the person's token
IF-05	06 Grounding checks	04 Answering agent	claims in, kept claims and failures out
IF-06	07 Guardrail	04 Answering agent	question screened, claims checked

Catalogue Mapping

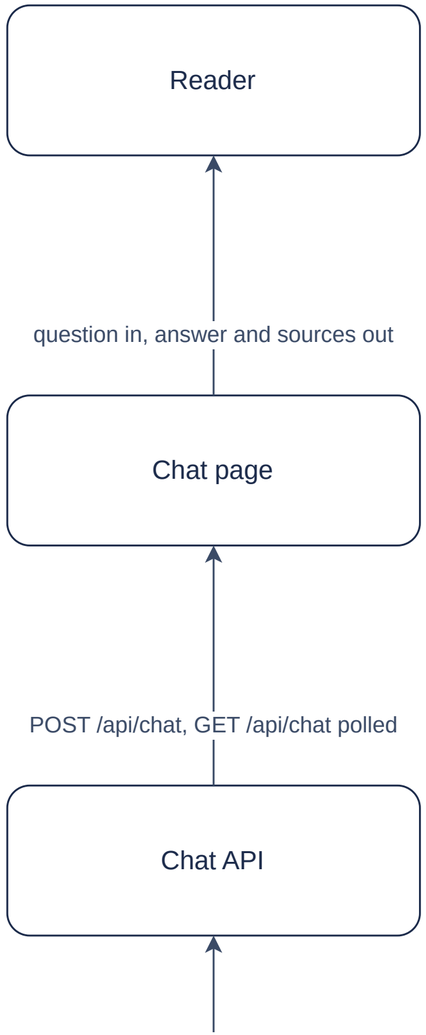
What each local participant stands for in the catalogue: an actor or an external system is a gap by design (no artefact of this catalogue realises a person or a vendor); a solution participant realises or partly realises a building block.

Local ID	Maps To	Relationship
01	gap	gap
02	ABB-018 Chat page	realises
03	ABB-019 Chat API	realises
04	ABB-014 Answering agent	realises
05	ABB-026 Tool gateway	partial
06	ABB-014 Answering agent	partial
07	gap	gap

The flow

UC-001 Ask a question and get a grounded answer: scenarios

S1 Main success scenario | S2 The sources cannot answer | S3 A table answers it



S1 Main success scenario

◀ Prev | Next ▶ | ▶ Play | Restart | Zoom to step: on

Arrow keys step, space plays

1. type a question and choose the collection
2. post the question with the last six turns
3. take the daily quota, invoke the agent as the person
4. screen the question
5. read the ontology rendition and the tables; search entities and passages
6. propose claims with citations
7. re-read every cited passage, cell and figure as the person
8. return the kept claims; send failures back once for repair
9. check each kept claim against its sources
10. return the rendered answer, sources, steps and cost
11. show the answer with numbered citations and the workbench

Data and measures

- The question, the history, the collection id
- The claims with citations, the sources, the steps, the usage and cost
- Term usage counters per collection, all time and per month